```
<!DOCTYPE html>
<html lang="tr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no">
    <title>Rick's Alien Tetris</title>
    <style>
        body { background: #000; color: #39ff14; font-family: sans-serif; display: flex; flex-direction: column; align-items: center; margin: 0; overflow: hidden; touch-action: manipulation; }
        #portal-bg { position: fixed; top: 50%; left: 50%; width: 500px; height: 500px; background: repeating-conic-gradient(from 0deg, #39ff14 0deg 20deg, #004400 20deg 40deg); filter: blur(60px); opacity: 0.1; border-radius: 50%; transform: translate(-50%, -50%); animation: rotate 10s linear infinite; z-index: -1; }
        @keyframes rotate { from { transform: translate(-50%, -50%) rotate(0deg); } to { transform: translate(-50%, -50%) rotate(360deg); } }
        canvas { border: 3px solid #39ff14; background: rgba(0,0,0,0.8); box-shadow: 0 0 15px #39ff14; width: 80vw; max-width: 300px; }
        .controls { display: grid; grid-template-columns: repeat(3, 1fr); gap: 10px; margin-top: 15px; width: 80vw; max-width: 300px; }
        button { padding: 15px; background: rgba(57, 255, 20, 0.1); border: 2px solid #39ff14; color: #39ff14; font-size: 20px; border-radius: 10px; outline: none; }
```

```
        button:active { background: #39ff14; color: #000; }
        #start-screen { position: fixed; inset: 0; background:
#000; display: flex; flex-direction: column; justify-content:
center; align-items: center; z-index: 100; }
    </style>
</head>
<body>
    <div id="portal-bg"></div>
    <div id="start-screen">
        <h1 style="text-shadow: 0 0 10px #39ff14;">RICK'S
ALIEN CRISIS</h1>
        <button onclick="startGame()">OYUNU BAŞLAT</button>
    </div>

    <div id="ui" style="text-align: center; margin-top: 10px;">
        <div id="score" style="font-size: 24px;">SKOR: 0</div>
        <div id="highscore" style="font-size: 16px; color:
#ffea00;">REKOR: 0</div>
    </div>

    <canvas id="tetris" width="240" height="400"></canvas>

    <div class="controls">
        <button onclick="move(-1)">⬅️</button>
        <button onclick="pRotate()">🔄</button>
        <button onclick="move(1)">➡️</button>
        <button style="grid-column: span 3;"
onclick="pDrop()">HIZLI DÜŞÜŞ ⬇️</button>
    </div>
```

```
<script>
    const canvas = document.getElementById('tetris');
    const context = canvas.getContext('2d');
    context.scale(20, 20);

    const yesSound = new Audio('https://
www.myinstants.com/media/sounds/rick-and-morty-
yes.mp3');
    const colors = [null, '#39ff14', '#00d4ff', '#ff00ff',
'#ffea00', '#ff4500', '#00ffcc', '#ff0055'];

    function createPiece(t) {
        if (t === 'T') return [[0, 1, 0], [1, 1, 1], [0, 0, 0]];
        if (t === 'O') return [[2, 2], [2, 2]];
        if (t === 'L') return [[0, 3, 0], [0, 3, 0], [0, 3, 3]];
        if (t === 'J') return [[0, 4, 0], [0, 4, 0], [4, 4, 0]];
        if (t === 'I') return [[0, 5, 0, 0], [0, 5, 0, 0], [0, 5, 0, 0], [0, 5,
0, 0]];
        if (t === 'S') return [[0, 6, 6], [6, 6, 0], [0, 0, 0]];
        if (t === 'Z') return [[7, 7, 0], [0, 7, 7], [0, 0, 0]];
    }

    const arena = Array.from({length: 20}, () =>
Array(12).fill(0));
    const player = { pos: {x: 0, y: 0}, matrix: null, score: 0 };

    function draw() {
        context.fillStyle = '#000';
        context.fillRect(0, 0, canvas.width, canvas.height);
        drawMatrix(arena, {x: 0, y: 0});
```

```
        drawMatrix(player.matrix, player.pos);
    }

    function drawMatrix(matrix, offset) {
        matrix.forEach((row, y) => {
            row.forEach((value, x) => {
                if (value !== 0) {
                    context.shadowBlur = 8;
                    context.shadowColor = colors[value];
                    context.fillStyle = colors[value];
                    context.fillRect(x + offset.x, y + offset.y, 1, 1);
                    context.shadowBlur = 0;
                }
            });
        });
    }

    function merge(arena, player) {
        player.matrix.forEach((row, y) => {
            row.forEach((value, x) => {
                if (value !== 0) arena[y + player.pos.y][x +
player.pos.x] = value;
            });
        });
    }

    function collide(arena, player) {
        const [m, o] = [player.matrix, player.pos];
        for (let y = 0; y < m.length; ++y) {
            for (let x = 0; x < m[y].length; ++x) {
```

```
                if (m[y][x] !== 0 && (arena[y + o.y] && arena[y + o.y]
[x + o.x]) !== 0) return true;
            }
        }
        return false;
    }

    function arenaSweep() {
        outer: for (let y = arena.length - 1; y > 0; --y) {
            for (let x = 0; x < arena[y].length; ++x) {
                if (arena[y][x] === 0) continue outer;
            }
            const row = arena.splice(y, 1)[0].fill(0);
            arena.unshift(row);
            ++y;
            player.score += 10;
            yesSound.play();
            if(navigator.vibrate) navigator.vibrate(100);
        }
    }

    function pDrop() {
        player.pos.y++;
        if (collide(arena, player)) {
            player.pos.y--;
            merge(arena, player);
            playerReset();
            arenaSweep();
            updateScore();
        }
```

```
        dropCounter = 0;
    }

    function move(dir) {
        player.pos.x += dir;
        if (collide(arena, player)) player.pos.x -= dir;
    }

    function pRotate() {
        const pos = player.pos.x;
        let offset = 1;
        rotate(player.matrix, 1);
        while (collide(arena, player)) {
            player.pos.x += offset;
            offset = -(offset + (offset > 0 ? 1 : -1));
            if (offset > player.matrix[0].length) {
                rotate(player.matrix, -1);
                player.pos.x = pos;
                return;
            }
        }
    }

    function rotate(matrix, dir) {
        for (let y = 0; y < matrix.length; ++y) {
            for (let x = 0; x < y; ++x) [matrix[x][y], matrix[y][x]] =
[matrix[y][x], matrix[x][y]];
        }
        if (dir > 0) matrix.forEach(row => row.reverse()); else
matrix.reverse();
```

```
    }

    function playerReset() {
        const pieces = 'ILJOTSZ';
        player.matrix = createPiece(pieces[pieces.length *
Math.random() | 0]);
        player.pos.y = 0;
        player.pos.x = (arena[0].length / 2 | 0) -
(player.matrix[0].length / 2 | 0);
        if (collide(arena, player)) {
            const high = JSON.parse(localStorage.getItem('hi'))
|| 0;
            if(player.score > high) localStorage.setItem('hi',
player.score);
            arena.forEach(row => row.fill(0));
            player.score = 0;
            updateScore();
        }
    }

    function updateScore() {
        document.getElementById('score').innerText = "SKOR:
" + player.score;
        document.getElementById('highscore').innerText =
"REKOR: " + (localStorage.getItem('hi') || 0);
    }

    let dropCounter = 0;
    let lastTime = 0;
    function update(time = 0) {
```

```
        const deltaTime = time - lastTime;
        lastTime = time;
        dropCounter += deltaTime;
        if (dropCounter > 1000) pDrop();
        draw();
        requestAnimationFrame(update);
    }

    function startGame() {
        document.getElementById('start-screen').style.display
= 'none';
        playerReset();
        updateScore();
        update();
    }
  </script>
</body>
</html>
```